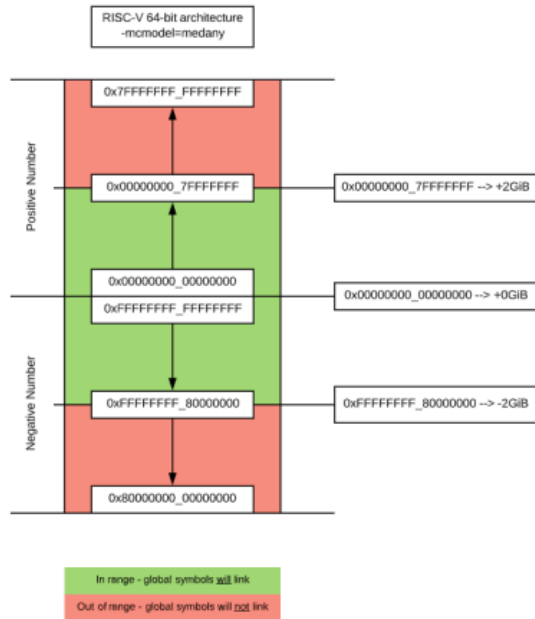


Analysing the make file and commands:

-mcmmodel=medany: Used for 64-bit architectures where code can be linked at any base address, but linked global symbols follow a similar +/- 2GiB range restriction.



```
riscv64-unknown-elf-gcc -g -I inc -c src/startup.s -o build/startup_asm.o
```

```
riscv64-unknown-elf-gcc -g -I inc -c src/trap_entry.s -o build/trap_entry_asm.o
```

```
riscv64-unknown-elf-gcc -Wall -mcmmodel=medany -g -I inc -O1 -c src/trap.c -o build/trap.o
```

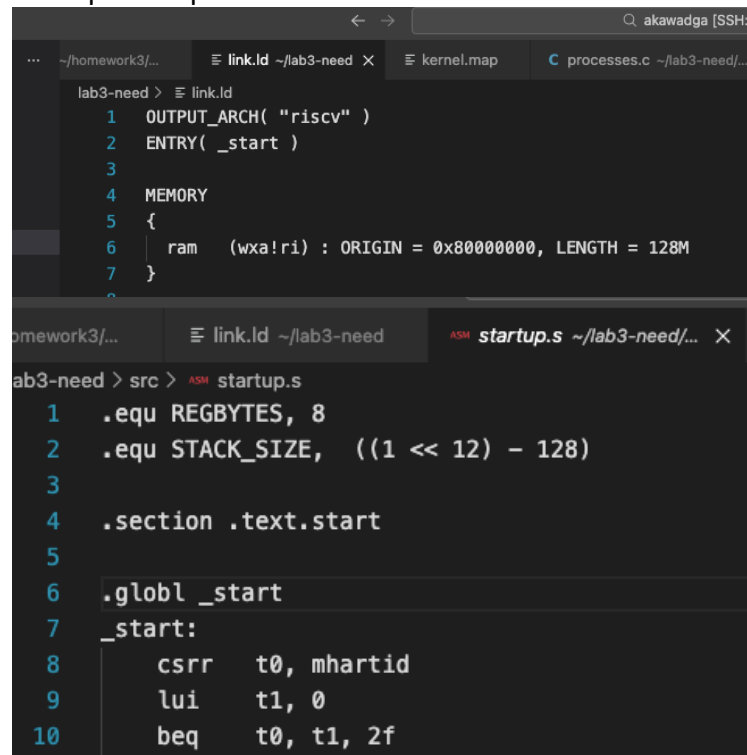
```
riscv64-unknown-elf-gcc -Wall -mcmmodel=medany -g -I inc -O1 -c src/uart.c -o build/uart.o
```

Reference: <https://starfivetech.com/uploads/riscv-large-code-model-workaround.pdf>

Debug session command: `riscv64-unknown-elf-gdb kernel.elf`

Code Flow and Understanding of Homework 3:

Stack pointer points to the start.c



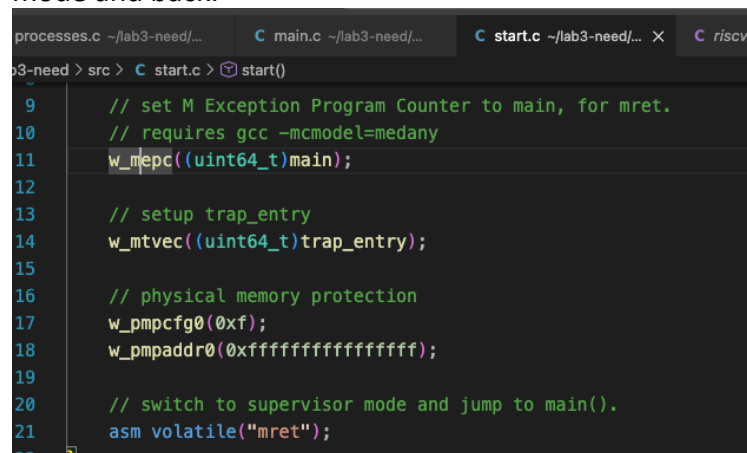
The screenshot shows a code editor with two tabs: `link.ld` and `startup.s`. The `link.ld` tab is active, showing the following assembly code:

```
1 OUTPUT_ARCH( "riscv" )
2 ENTRY( _start )
3
4 MEMORY
5 {
6   ram (wxa!ri) : ORIGIN = 0x80000000, LENGTH = 128M
7 }
```

The `startup.s` tab is also visible, showing the following assembly code:

```
1 .equ REGBYTES, 8
2 .equ STACK_SIZE, ((1 << 12) - 128)
3
4 .section .text.start
5
6 .globl _start
7 _start:
8     csrr    t0, mhartid
9     lui     t1, 0
10    beq     t0, t1, 2f
```

The stack pointer has set the machine exception pointer and the vector for main function. It will help us later during the execution of processes and switching modes from machine to user mode and back.



The screenshot shows a code editor with two tabs: `start.c` and `riscv.c`. The `start.c` tab is active, showing the following C code:

```
9 // set M Exception Program Counter to main, for mret.
10 // requires gcc -mmodel=medany
11 w_mepc((uint64_t)main);
12
13 // setup trap_entry
14 w_mtvec((uint64_t)trap_entry);
15
16 // physical memory protection
17 w_pmpcfg0(0xf);
18 w_pmpaddr0(0xffffffffffffffff);
19
20 // switch to supervisor mode and jump to main().
21 asm volatile("mret");
```

mret => will help us jump into the main.c

```
processes.c ~/lab3-need/...  C main.c ~/lab3-need/... X  C start.c ~/lab3-need/...
lab3-need > src > C main.c > main()
1  #include "uart.h"
2  #include "scheduler.h"
3
4  int main() {
5      uart_init();
6      print_s("Before scheduler_init\n");
7      scheduler_init();
8      print_s("Before ecall\n");
9  }
```

The main function consists of `uart_init` where it goes through a routine which enables the UART device to work. UART is a physical hardware device which enables us to transmit and receive serial data.

Reference: <https://twilco.github.io/riscv-from-scratch/2019/07/08/riscv-from-scratch-3.html#:~:text=UART%20stands%20for%20%E2%80%9CUniversal%20Asynchronous,microcontroller%2C%20our%20virt%20machine%20included.>

It helps us ensure that the hardware is pointing to the correct MMIO address.

The **scheduler_init()** calls the function in **scheduler.c** which sets the states of the 3 processes as 0 [ready to run state] and helps save the address of the 3 processes by using pointers.

```
lab3-need > src > C scheduler.c > scheduler_init()

7  uintptr_t addr1, addr2, addr3, ret_addr;
8  uint8_t ecall_id, proc;
9
10 void scheduler_init() {
11     state1 = 0;
12     state2 = 0;
13     state3 = 0;
14     addr1 = (uintptr_t)process1;
15     addr2 = (uintptr_t)process2;
16     addr3 = (uintptr_t)process3;
17     ecall_id = 0;
18     proc = 0;
19 }
20
```

We are back to **main.c**; print the "Before ecall". The **asm volatile** helps the compiler to not optimize the instruction command. Consequently, we utilise **ecall**.

Reference: <https://gcc.gnu.org/onlinedocs/gcc-12.1.0/gcc/Extended-Asm.html#:~:text=Remarks,and%20add%20to%20dst%20.>

```
int main() {
    uart_init();
    print_s("Before scheduler_init\n");
    scheduler_init();
    print_s("Before ecall\n");

    // Calling the scheduler here by ecall
    asm volatile("ecall");
    print_s("After ecall\n");
}
```

The `ecall` pulls the flow into machine mode and calls `trap_entry.s` where the stack pointer is moved by 256 bytes to make room for the new values which need to be saved into the registers.

```
lab3-need > src > trap_entry.s
4  trap_entry:
20      sd a2, 88(sp)
21      sd a3, 96(sp)
22      sd a4, 104(sp)
23      sd a5, 112(sp)
24      sd a6, 120(sp)
25      sd a7, 128(sp)
26      sd s2, 136(sp)
27      sd s3, 144(sp)
28      sd s4, 152(sp)
29      sd s5, 160(sp)
30      sd s6, 168(sp)
31      sd s7, 176(sp)
32      sd s8, 184(sp)
33      sd s9, 192(sp)
34      sd s10, 200(sp)
35      sd s11, 208(sp)
36      sd t3, 216(sp)
37      sd t4, 224(sp)
38      sd t5, 232(sp)
39      sd t6, 240(sp)
40
41      call handle_trap
42
```

The `handle_trap` in `trap_entry.s` calls `schedule()` in `schedule.c`, where the current values are as follows:

- `ecall_id = 0`
- `proc = 0`
- `state1 = 0`

The if condition to check whether 1st process's state is 0 is placed.

```
lab3-need > src > schedule.c > schedule()
21  void schedule(){
54      if (state1 == 0) {
55          proc = 1;
56          w_mepc(addr1);
57      } else if (state2 == 0) {
58          proc = 2;
59          w_mepc(addr2);
60      } else if (state3 == 0) {
61          proc = 3;
62          w_mepc(addr3);
```

We then set the process as 1 and utilize the control and status register to point to the 1st process's address.

```
Continuing.  
Breakpoint 11.1, w_mepc (x=2147484544) at inc/riscv.h:27  
27      asm volatile("csrw mepc, %0" : : "r"(x));  
(gdb)
```

After restoring register values, mret helps us enter user mode again and enter process1() in processes.c.

```
link.ld  
Makefile  
.gdb_history  
OUTLINE  
TIMELINE  
SSH: remoteserver-check 0 0 6  
(gdb) n  
76      addi sp, sp, 256  
(gdb) n  
trap_entry () at src/trap_entry.s:78  
78      mret  
(gdb) n  
process1 () at src/processes.c:6  
6      void process1(){  
(gdb)
```

Process 1 OUTPUT:

```
Continuing on signal 15 from pid 234455 (ssh/ssh)  
[akawadga@bondi ~/lab3-need]$ qemu-system-riscv64 -machine virt -nographic -bios none -kernel kernel.elf -S -gdb tcp::12222  
Before scheduler_init  
Before ecall  
process1 is running
```

Since t6 register was used to save the values of the state of the processes in exit(), we shall utilise the same register and save the value as 2 [yield state] to align with the requirement.

```
(gdb) n  
8      yield();  
(gdb) s  
yield_ () at inc/stdlib.h:10  
10      asm volatile("li t6, 2"); // We use t6 to store the ecall_id  
(gdb)
```

ecall trigger the trap_entry again.

```
Continuing.  
Breakpoint 9, trap_entry () at src/trap_entry.s:6  
6      addi sp, sp, -256  
(gdb)
```

Since the instruction takes 4 bytes in our emulator of RISC-V 64 bit architecture, we shall increment the return address/ program counter by 4 so that we are able to execute the next process by pointing it to next process's stack pointer.

We have also marked the state1 to 2 (yield state).

```
Breakpoint 6, schedule () at src/scheduler.c:27
27      if(ecall_id == 1 && proc == 1) state1 = 1;
(gdb) n
31      if(ecall_id == 2){
(gdb) n
32          asm volatile("csrr %0, mepc" : "=r" (ret_addr);
(gdb) n
33          ret_addr += 4;
(gdb) n
34          if(proc == 1){
(gdb) n
35              state1 = 2;
(gdb) n
36              addr1 = ret_addr;
(gdb) n
47      if(state1 == 1 && state2 == 1 && state3 == 1){
(gdb) n
57      } else if (state2 == 0) {
(gdb) n
58          proc = 2;
(gdb) n

Breakpoint 11.2, w_mepc (x=2147484588) at inc/riscv.h:27
27      asm volatile("csrw mepc, %0" : : "r"(x);
(gdb) █
```

The state2,3 /process2, 3 will proceed in the same iteration. Since we have not designed a particular scheduling technique, the processes will go in sequence as per the if-else statement flow.

Flushing the old register values:

```
Breakpoint 3, process2 () at src/processes.c:14
14      print_s("process2 is running\n");
(gdb) c
Continuing.

Breakpoint 9, trap_entry () at src/trap_entry.s:6
6      addi sp, sp, -256
(gdb) █
```

Process 2 printed after going through the same loop:

```
qemu-system-riscv64: terminating on signal 15 from pid 2944695 (-bin/csh)
lakawadga@bondi ~/lab3-need$ qemu-system-riscv64 -machine virt -nographic -bios none -kernel kernel.elf -S -gdb tcp::12222
Before scheduler_init
Before ecall
process1 is running
process2 is running
```

Same loop for process 3:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 6
Continuing.

Breakpoint 6, schedule () at src/scheduler.c:27
27     if(ecall_id == 1 && proc == 1) state1 = 1;
(gdb) c
Continuing.

Breakpoint 11.3, w_mepc (x=2147484632) at inc/riscv.h:27
27     asm volatile("csrw mepc, %0" : : "r"(x));
(gdb) c
Continuing.

Breakpoint 4, process3 () at src/processes.c:21
21     print_s("process3 is running\n");
(gdb) c
Continuing.

Breakpoint 9, trap_entry () at src/trap_entry.s:6
6     addi sp, sp, -256
(gdb) c
Continuing.

Breakpoint 10, handle_trap () at src/trap.c:7
7     schedule();
(gdb) c
Continuing.

Breakpoint 6, schedule () at src/scheduler.c:27
27     if(ecall_id == 1 && proc == 1) state1 = 1;
(gdb) c
Continuing.

Breakpoint 7, schedule () at src/scheduler.c:47
47     if(state1 == 1 && state2 == 1 && state3 == 1){
(gdb) c
Continuing.

Breakpoint 11.4, w_mepc (x=2147484566) at inc/riscv.h:27
27     asm volatile("csrw mepc, %0" : : "r"(x));
(gdb) █
```

```
Breakpoint 10, handle_trap () at src/trap.c:7
7      schedule();
(gdb) c
Continuing.

Breakpoint 6, schedule () at src/scheduler.c:27
27      if(ecall_id == 1 && proc == 1) state1 = 1;
(gdb) c
Continuing.

Breakpoint 7, schedule () at src/scheduler.c:47
47      if(state1 == 1 && state2 == 1 && state3 == 1){
(gdb) c
Continuing.

Breakpoint 11.6, w_mepc (x=2147484654) at inc/riscv.h:27
27      asm volatile("csrw mepc, %0" : : "r"(x));
(gdb) c
Continuing.

Breakpoint 9, trap_entry () at src/trap_entry.s:6
6      addi sp, sp, -256
(gdb) c
Continuing.

Breakpoint 10, handle_trap () at src/trap.c:7
7      schedule();
(gdb) c
Continuing.

Breakpoint 6, schedule () at src/scheduler.c:27
27      if(ecall_id == 1 && proc == 1) state1 = 1;
(gdb) c
Continuing.

Breakpoint 7, schedule () at src/scheduler.c:47
47      if(state1 == 1 && state2 == 1 && state3 == 1){
(gdb) c
Continuing.
```

The `exit_()` also goes through the same flow as `yield()`, however, the `t6` register value is saved as 1 (Process has terminated).

```
Breakpoint 4, process3 () at src/processes.c:21
21      print_s("process3 is running\n");
(gdb) c
Continuing.

Breakpoint 9, trap_entry () at src/trap_entry.s:6
6      addi sp, sp, -256
(gdb)
Continuing.

Breakpoint 10, handle_trap () at src/trap.c:7
7      schedule();
(gdb)
Continuing.

Breakpoint 6, schedule () at src/scheduler.c:27
27      if(ecall_id == 1 && proc == 1) state1 = 1;
(gdb) c
Continuing.

Breakpoint 14, schedule () at src/scheduler.c:33
33      ret_addr += 4;
(gdb)
Continuing.

Breakpoint 7, schedule () at src/scheduler.c:47
47      if(state1 == 1 && state2 == 1 && state3 == 1){
(gdb)
Continuing.

Breakpoint 11.4, w_mepc (x=2147484566) at inc/riscv.h:27
27      asm volatile("csrw mepc, %0" : : "r"(x));
(gdb)
Continuing.

Breakpoint 15, exit_ () at inc/stdlib_.h:5
5      asm volatile("li t6, 1"); // We use t6 to store the ecall_id
(gdb) c
```

FINAL OUTPUT:

```
[akawadga@bondi ~/lab3-need]$ qemu-system-riscv64 -machine virt -nographic -bios none -kernel kernel.elf -S -gdb tcp::12222
Before scheduler_init
Before ecall
process1 is running
process2 is running
process3 is running
process1 is back from yield
process2 is back from yield
process3 is back from yield
All done.
```

Debugging Observation: If we remove `exit()` from all three processes, no processes are going to exit and process 1 and 2 will keep satisfying the if conditions consequently allowing the `main.c` to take over and print the "After ecall" and run that loop.

```
Run till exit from #0 print_s (s=0x80000531 "all\n", s@entry=0x80000528 "Before ecall\n") at src/uart.c:29
main () at src/main.c:11
11     asm volatile("ecall");
(gdb) n

Breakpoint 6, trap_entry () at src/trap_entry.s:41
41     call handle_trap
(gdb) n

Breakpoint 7, handle_trap () at src/trap.c:7
7     schedule();
(gdb) s

Breakpoint 3, schedule () at src/scheduler.c:25
25     asm volatile("mv %0, t6" : "=r" (ecall_id));
(gdb) s
27     if(ecall_id == 1 && proc == 1) state1 = 1;
(gdb) s
31     if(ecall_id == 2){
47         if(state1 == 1 && state2 == 1 && state3 == 1){
(gdb) s
54     if (state1 == 0) {
(gdb) s
55         proc = 1;
(gdb) s
56         w_mepc(addr1);
(gdb) s
w_mepc (x=2147484544) at inc/riscv.h:27
27     asm volatile("csrw mepc, %0" : : "r"(x));
(gdb) █

[jakawadga@bondi ~/lab3-need]$ make run
rm -f build/*.asm.o build/*.o kernel.elf kernel.img
riscv64-unknown-elf-gcc -g -I inc -c src/startup.s -o build/startup_asm.o
riscv64-unknown-elf-gcc -g -I inc -c src/trap_entry.s -o build/trap_entry_asm.o
riscv64-unknown-elf-gcc -Wall -mcmodel=medany -g -I inc -O1 -c src/trap.c -o build/trap.o
riscv64-unknown-elf-gcc -Wall -mcmodel=medany -g -I inc -O1 -c src/uart.c -o build/uart.o
riscv64-unknown-elf-gcc -Wall -mcmodel=medany -g -I inc -O1 -c src/scheduler.c -o build/scheduler.o
riscv64-unknown-elf-gcc -Wall -mcmodel=medany -g -I inc -O1 -c src/processes.c -o build/processes.o
riscv64-unknown-elf-gcc -Wall -mcmodel=medany -g -I inc -O1 -c src/main.c -o build/main.o
riscv64-unknown-elf-gcc -Wall -mcmodel=medany -g -I inc -O1 -c src/start.c -o build/start.o
riscv64-unknown-elf-ld -T link.ld -o kernel.elf build/startup_asm.o build/trap_entry_asm.o build/trap.o build/uart.o build/scheduler.o build/processes.o build/main.o build/start.o
objcopy kernel.elf -I binary kernel.img
qemu-system-riscv64 -M virt -kernel kernel.img -bios none -serial stdio -display none
Before scheduler_init
Before ecall
process1 is running
process2 is running
process3 is running
process1 is back from yield
process2 is back from yield
process1 is back from yield
After ecall
█
```

Kill the running debug session process.

```
[Inferior 1 (process 1) detached]
[jakawadga@bondi ~/lab3-need]$ ps aux | grep qemu-system-riscv64
shengloc 1001415 64.0 0.8 1704728 1096180 ? Ssl Jan19 22820:00 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel hello.img -s -S
chenangl 1421138 0.0 0.0 1704804 48664 ? Ssl Jan22 0:35 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel trap.img -gdb tcp::23234 -S
hongduyp 1008630 0.0 0.0 1704804 49416 ? Ssl Jan23 1:24 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel trap.img -gdb tcp::5986 -S
chenangl 1622019 0.0 0.0 1704736 48632 ? Ssl Jan23 0:31 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel hello.img -gdb tcp::15342 -S
cmeyn 1623293 0.0 0.0 1704728 50284 ? Ssl Jan23 0:30 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel program.img -gdb tcp::18356 -S
wenkuns 2038494 99.7 0.0 1704804 50260 ? Ssl Jan25 27162:04 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel trap.img -gdb tcp::51280 -S
melindmt 2193975 0.0 0.0 1704804 49968 ? Ssl Jan26 0:29 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel trap.img -gdb tcp:0 -S
melindmt 2194369 0.0 0.0 1704796 49184 ? Ssl Jan26 0:27 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel trap.img -gdb tcp:0 -S
melindmt 2195267 0.0 0.0 1704804 52204 ? Ssl Jan26 0:25 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel trap.img -gdb tcp:0 -S
melindmt 2199646 0.0 0.0 1704804 49516 ? Ssl Jan26 0:27 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel trap.img -gdb tcp:0 -S
melindmt 2210631 0.0 0.0 1704804 49456 ? Ssl Jan26 0:30 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel trap.img -gdb tcp:0 -S
melindmt 2272889 0.0 0.0 1704804 49752 ? Ssl Jan27 0:28 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel trap.img -gdb tcp:0 -S
melindmt 2276290 0.0 0.0 1704804 49440 ? Ssl Jan27 0:24 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel trap.img -gdb tcp:0 -S
melindmt 2276748 0.0 0.0 1704804 50288 ? Ssl Jan27 0:28 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel trap.img -gdb tcp:0 -S
melindmt 2288886 0.0 0.0 1704804 49412 ? Ssl Jan27 0:26 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel trap.img -gdb tcp:0 -S
hongduyp 2697386 96.9 0.0 1704828 50828 ? Ssl Feb12 1205:59 qemu-system-riscv64 -M virt -kernel kernel.img -bios none -serial stdio -display none -gdb tcp::1270 -S
hongduyp 2709537 0.0 0.0 1704828 51496 ? Ssl Feb12 0:01 qemu-system-riscv64 -M virt -kernel kernel.img -bios none -serial stdio -display none -gdb tcp::598618 -S
hongduyp 2709899 0.0 0.0 1704828 50496 ? Ssl Feb12 0:01 qemu-system-riscv64 -M virt -kernel kernel.img -bios none -serial stdio -display none -gdb tcp::1234555 -S
akawadga 2953668 1.2 0.0 1702656 45404 pts/8 Ssl+ 16:19 0:03 qemu-system-riscv64 -machine virt -nographic -bios none -kernel kernel.elf -S -gdb tcp::12222
akawadga 2954354 0.0 0.0 16480 1268 pts/18 S+ 16:23 0:00 grep --color=auto qemu-system-riscv64
[jakawadga@bondi ~/lab3-need]$ kill 2953668
[jakawadga@bondi ~/lab3-need]$ █
```